

프로토콜 액션을 실제 처리 함수에 잇는다

"RemoteStartTransaction"



`remote_handler(payload)`

목표 — 프로토콜 액션과 실제 처리 함수를 **정적으로** 연결한다

직접 호출은 보이지만, 간접 호출은 대상이 숨는다

직접 호출 - 대상이 보임

```
on_message()  
  | 직접 호출  
  ▼  
remote_handler()
```

콜그래프에 **간선**이 그대로 있다

간접 호출 - 대상이 숨음

```
on_message()  
  |  
  ▼  
entry.fn(payload) // 실제 대상 = ?
```

```
table[ACTION_REMOTE_START].fn = remote_handler;
```

연결은 호출식이 아니라 **데이터(등록부)**에 있다

발표 내내 쓸 하나의 예제

```
typedef struct {
    int      action;
    Handler  fn;
} Entry;

static Entry table[] = {
    { ACTION_REMOTE_START, remote_handler },
    { ACTION_STOP,         stop_handler   }
};

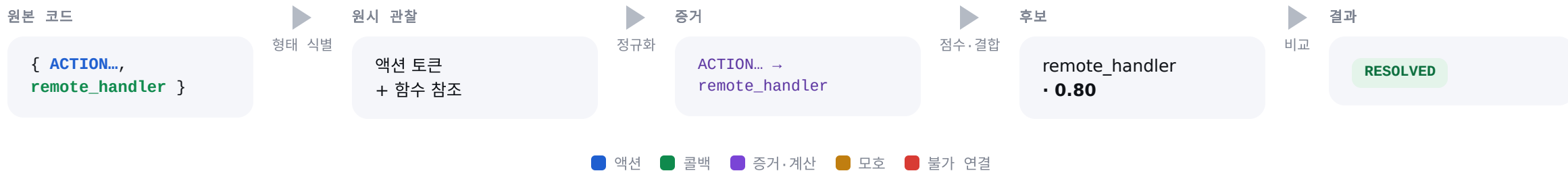
void dispatch(Frame *f) {
    table[f->action].fn(f->payload);    // 간접 호출
}
```

■ 액션 식별자 ■ 콜백 함수 ■ 간접 호출

호출식은 함수를 이름으로 부르지 않는다



코드가 단계마다 구체적 형태로 바뀐다



이후 슬라이드는 이 지도의 각 화살표를 하나씩 확대한다

같은 초기화 항목의 두 자식을 한 쌍으로 읽는다

```
{ ACTION_REMOTE_START, remote_handler }
```

▼
AST로 관찰

```
initializer  
├─ element[0]: ACTION_REMOTE_START  
└─ element[1]: remote_handler
```

▼
추출

RawRegistration

action	ACTION_REMOTE_START
callback	remote_handler
site	table 초기화 항목

두 값이 같은 초기화 항목의 자식이므로 하나의 등록 쌍으로 해석한다

필드 이름으로 짝을 찾되, 같은 객체 안에서만

허용 - 같은 객체

```
{  
  .action = ACTION_REMOTE_START,  
  .fn      = remote_handler  
}
```



ACTION_REMOTE_START → remote_handler

같은 부모 객체 + 의미 필드 action/fn → 한 쌍

거부 - 서로 다른 항목

```
table[i].action = ACTION_REMOTE_START;  
table[j].fn      = remote_handler;
```



i == j 증명 불가 → 연결 안 함

부모가 같다고 증명되지 않으면 쌍으로 묶지 않는다

같은 수신자-같은 인덱스면 같은 슬롯이다

허용 - 같은 슬롯 k

```
table[k].action = ACTION_REMOTE_START;  
table[k].fn      = remote_handler;
```

수신자 table + 인덱스 k 동일 → 같은 논리 슬롯



ACTION_REMOTE_START → remote_handler

거부 - 다른 인덱스

```
table[k].action = ACTION_REMOTE_START;  
table[m].fn      = remote_handler;
```

k 와 m 이 같다고 증명되지 않음



UNRESOLVED · 증거 없음

등록 함수는 이름이 아니라 저장까지 따라가야 한다

호출부

```
register_handler(  
    ACTION_REMOTE_START,  
    remote_handler  
);
```

피호출부 - 인자를 따라간다

```
void register_handler(int action, Handler fn) {  
    table[next].action = action;    // 최종 저장  
    table[next].fn      = fn;  
}
```

인자 흐름

ACTION_REMOTE_START → 파라미터 action → 저장 table[next].action
remote_handler → 파라미터 fn → 저장 table[next].fn



ACTION_REMOTE_START → remote_handler

최종 쌍 저장을 못 찾으면 → REGISTRAR_STORE_NOT_REACHED

switch 분기에서는 액션과 호출이 바로 붙어 있다

```
switch (frame->action) {  
    case ACTION_REMOTE_START:  
        remote_handler(frame->payload);  
        break;  
}
```

같은 분기

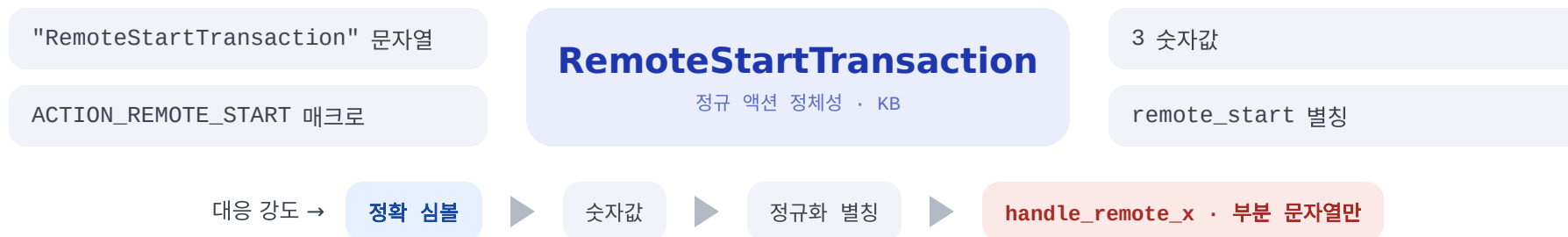
case ACTION_REMOTE_START

call remote_handler()

ACTION_REMOTE_START → remote_handler

동일한 case 분기 안에서 액션 선택과 호출이 직접 결합되어 있다

한 액션의 여러 표기를 하나의 정체성으로 모은다



액션을 ‘명시’하는가, ‘답은’ 것뿐인가

후보 A

ACTION_REMOTE_START

KB 정확 심볼 → 강한 대응 · 프로토콜 액션을 명시적으로 식별

후보 B

handle_remote_x

텍스트만 겹침 → 약한 대응 · 어떤 액션인지 명시하지 않음

둘 다 ‘원격 시작’을 **암시**할 수 있으나, **명시적으로 식별**하는 건 하나뿐이다

코드 조각 하나하나가 증거의 한 필드가 된다



같은 색이 어디서 왔는지를 보여준다 — 액션 토큰·콜백 심볼·공유 출처가 각 필드로

추출기가 직접 정하면 충돌을 못 푼다

직접 판정 — 충돌

등록 추출기 → foo



디스패치 추출기 → bar

누가 진실을 소유하는가? 판단 불가

증거 방식 — 공통 판정

등록 관찰 → Evidence A

디스패치 관찰 → Evidence B



공통 계산 → 하나의 판정

추출기는 **사실만 보고**하고, 충돌 해결은 **공통 판정 계층**이 수행한다

점수는 두 조건이 모두 강해야 강해진다

조건 1 · 구조

코드 구조가 믿을 만한 **액션→콜백** 관계인가? Prior(kind)

×

조건 2 · 식별자

식별자가 이 액션을 **명시적으로** 가리키나? match

0.80×1.00		0.80
0.80×0.00		0.00
0.70×1.00		0.70

구조와 액션 대응 중 하나라도 약하면 전체 증거도 제한된다

관계를 얼마나 직접 표현하느냐가 사전확률이다

<code>strcmp(action, "RemoteStart...")==0 → remote_handler()</code>	STRING_DISPATCH		0.90
<code>case ACTION_REMOTE_START: remote_handler()</code>	ENUM_CASE		0.85
<code>{ ACTION_REMOTE_START, remote_handler } // 호출은 딴 곳</code>	REGISTRATION		0.80
<code>register(ACTION..., remote_handler) // 피호출부 확인</code>	REGISTRAR		0.70
<code>함수 이름: handle_remote_start // 구조 연결 없음</code>	NAME_MATCH		0.70

위로 갈수록 직접 표현, 아래로 갈수록 해석이 더 필요 · 값은 정책값이지 보정된 확률이 아니다

한 등록을 세 방식으로 본 것뿐이다

```
{ ACTION_REMOTE_START, remote_handler }
```

같은 출처(provenance) 하나

table-entry 관찰 · 0.80
 handler-reference 관찰 · 0.80
 symbol-resolution 관찰 · 0.80

셋을 다 더하면



한 등록이 세 증인처럼 부풀려짐

MAX 하나로



같은 출처는 한 번만 센다 (provenance grouping)

세 레코드는 **세 증인이 아니라** 하나의 등록을 세 방식으로 본 것이다

서로 다른 곳의 확인은 신뢰를 올린다

출처 A · 등록부

```
{ ACTION_REMOTE_START, remote_handler }
```

점수 0.80 · 남은 의심 0.20

출처 B · SWITCH

```
case ACTION_REMOTE_START:
    remote_handler();
```

점수 0.85 · 남은 의심 0.15

남은 의심 = $0.20 \times 0.15 = 0.03$

신뢰도 = $1 - 0.03 = 0.97$

독립된 두 출처가 확인 → 남은 의심을 곱한다 (noisy-OR)

같은 두 증거 $0.80 \cdot 0.49$ — 방식마다 결과가 다르다

MAX → 0.80

확인을 무시한다

SUM → 1.29

유효 범위를 넘어 과다 계산

AVERAGE → 0.645

증거가 늘수록 오히려 낮아짐

NOISY-OR → 0.898

유계 + 독립 확증을 보상

약한 이름 셋이 구조적 증명이 될 순 없다

약한 이름 관찰 3개

0.70 · 0.70 · 0.70



noisy-OR 그대로면 거의 확신? — 아니오

weak-only cap 적용

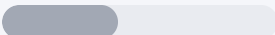
정확/해석된 식별자 증거가 하나도 없으면 →



강한 증거가 아무리 많아도 → 절대 확신엔 못 미친다 global cap 0.99

두 가지 질문으로 확정·모호·미해결을 가른다

질문 1 · 1위가 충분히 강한가? **floor 0.50**

top  0.42

0.42 < 0.50 → UNRESOLVED

질문 2 · 2위보다 뚜렷이 앞서나? **margin 0.15**

top  0.66

2nd  0.56

차이 0.10 < 0.15 → AMBIGUOUS

둘 다 통과

top  0.72

2nd  0.56

차이 0.16 ≥ 0.15 → RESOLVED

보이는 공백이 보이지 않는 오탐보다 안전하다

잘못 확정하면



미해결로 두면



한 액션에 후보가 둘 — 어느 쪽을 고를까

```
static Reg table[] = {  
    { ACTION_DATA_TRANSFER, foo }    // 등록  
};  
  
void dispatch(Frame *f) {  
    switch (f->action) {  
        case ACTION_DATA_TRANSFER:  
            bar(f->payload);          // 디스패치  
    }  
}
```

후보 foo

?

후보 bar

원본이 두 갈래 증거로 갈라진다

경로 A · 등록

```
{ ACTION_DATA_TRANSFER, foo }
```



Evidence A

kind	REGISTRATION_INIT
action	ACTION_DATA_TRANSFER
callback	foo

경로 B · SWITCH

```
case ACTION_DATA_TRANSFER:
    bar(f->payload);
```



Evidence B

kind	ENUM_CASE
action	ACTION_DATA_TRANSFER
callback	bar

각 값이 어느 소스 요소에서 나왔는지 따라간다

foo · REGISTRATION_INIT

prior 0.80 × match HEURISTIC_SUBSTRING 0.70



bar · ENUM_CASE

prior 0.85 × match NORMALIZED_NAME 0.85



디스패치(bar)의 식별자 대응이 더 강해 **점수가 더 높다** — 종류가 아니라 대응 강도의 차이

두 검사를 모두 통과 → **bar** 로 확정



floor · $0.72 \geq 0.50$ ✓

margin · $0.72 - 0.56 = 0.16 \geq 0.15$ ✓

RESOLVED → bar

입력을 바꾸면 결과가 이렇게 달라진다

A · switch 제거

foo 0.56 단독, 경쟁 없음

RESOLVED → foo

B · 점수가 근접

bar 0.66 · foo 0.56 (차이 0.10)

AMBIGUOUS

C · 약한 증거만

결합이 0.85를 넘어도

weak-only cap → 0.85

D · 구조적 쌍 없음

후보 자체가 없음

UNRESOLVED + 사유

판정의 각 요소가 출력 JSON의 필드가 된다

RESOLVED → bar

판정 결과
1위 후보
결합 점수
모든 대안
소스 레코드



직렬화

handler_resolutions	
status	← 판정 결과
chosen	← 1위 후보
confidence	← 결합 점수
candidates	← 모든 대안
evidence	← 소스 레코드
unresolved	← 실패 사유

확정된 핸들러가 오염 분석의 출발점이 된다

확정된 핸들러

```
int remote_handler(Payload *p) {
    int limit = p->chargingLimit;
    set_current_limit(limit);
}
```

프로토콜 필드 `chargingLimit`

지역 변수 `limit`

싱크 `set_current_limit`

핸들러가 미해결이면

미해결

신뢰할 소스 루트 없음

이 액션의 하위 분석 중단

핸들러가 없으면 오염의 출발점이 없어 분석이 멈춘다

정적으로 슬롯을 증명할 수 있는가가 경계다

지원 — 같은 슬롯

```
table[k].action = ACTION;  
table[k].fn      = fn;
```

같은 수신자 + 같은 슬롯 → 쌍 복원 가능

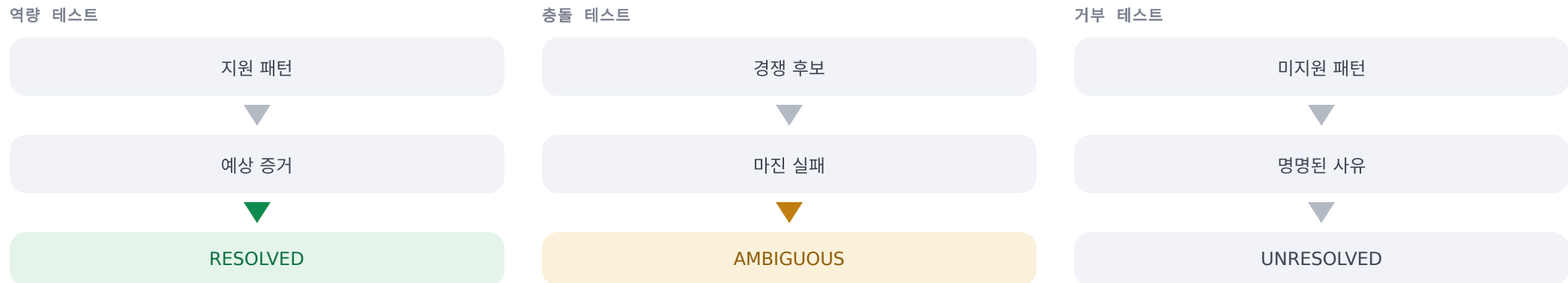
미지원 — 런타임 탐색

```
for (...) {  
  if (table[i].action == action)  
    table[i].fn = fn;  
}
```

런타임 탐색 + 심볼릭 인덱스 → 루프·값 추론 필요 → **미해결 · 사유 명명**

전체 지원/미지원 목록은 **부록**으로 — 여기선 경계의 **이유**를 그림으로

테스트는 '구현 동작'을 고정하지, '현실 정확도'를 증명하진 않는다



고정 픽스처 테스트 = 구현 동작 증명 · 임의 펌웨어의 현실 정확도는 별개

한 예제로 보는 전체 사슬 — 단계마다 구체적 객체



단계 이름의 나열이 아니라, **각 단계에 실제로 존재하는 객체**를 보여준다

부록 — 상수 · 사유 · 스키마

사전확률 · 매치 배수 · 임계값

STRING_DISPATCH 0.90 · ENUM_CASE 0.85 REGISTRATION_* 0.80 · REGISTRAR
0.70 · NAME 0.70/0.65 — EXACT 1.00 · RESOLVED_VALUE 0.95 NORMALIZED
0.85 · SUBSTRING 0.70 · NONE 0.00 — floor 0.50 · margin 0.15 · weak-
only cap 0.85 global cap 0.99 · registrar depth 2

미해결 사유 · 출력

NO_EVIDENCE · LOW_CONFIDENCE REGISTRAR_STORE_NOT_REACHED
REGISTRAR_SEARCH_THEN_WRITE EXTERNAL_DEFINITION · MISSING_POINTSTO
DYNAMIC_ACTION_ID · REGISTRATION_OUT_OF_TU — handler_resolutions
(권위) · handler_maps (호환) limitations (범위 한계)

본문은 변환의 이해에 집중 · 수치·전체 분류는 여기에만